

BuySellChain

Sistema d'Asta Trustless su Blockchain Hyperledger Fabric

Documento di Progettazione e Analisi di Sicurezza

Salvucci Franco (Mat. 0368692)
Ionut Georgian Zbirciog (Mat. 0381279)

12 Marzo 2026

Sommario

Il mercato immobiliare tradizionale è caratterizzato da inefficienze e dipendenza da intermediari. BuySellChain propone un sistema decentralizzato basato su **Blockchain** per eliminare l'agente immobiliare fisico. L'architettura su Hyperledger Fabric gestisce tramite Smart Contract la validazione delle offerte e il trasferimento della proprietà digitale. Il sistema garantisce trasparenza assoluta e azzeramento delle commissioni.

Indice

1	Introduzione	2
2	Architettura del Sistema	3
2.1	Gestione dell'Identità e Sicurezza	4
2.2	User Requirements Definition	5
2.3	Frontend	7
2.4	Backend (Security Gateway)	8
2.4.1	Authentication & Authorization	8
2.4.2	Modello Dati & Interfaccia RESTful API	9
2.4.3	Logica di Bussiness	15
2.5	Blockchain Layer	17
2.6	Database Off-Chain	18
3	Superficie osservabile e Perimetro di Attacco	19
3.1	Livello Client (Frontend)	19
3.2	Livello Application / Security Gateway (Backend)	20
3.3	Livello Dati Off-Chain (Database PostgreSQL)	20
4	Possibili Attacchi e Contromisure	22
4.1	Difesa del Frontend e Gestione Sicura delle Sessioni	22
4.2	Hardening del Security Gateway (Backend)	23
4.3	Protezione del Database Off-Chain e dell'Ambiente Docker	23
	Bibliografia	25

Capitolo 1

Introduzione

Il sistema **BuySellChain** mira a decentralizzare il mercato immobiliare tramite un servizio di aste online. Come verrà illustrato, l'agente si limita alla registrazione dell'asta e non svolge più il ruolo di intermediario tipico del modello tradizionale. In questa architettura, l'intermediazione è affidata alla Blockchain, incaricata di validare ogni offerta. Questo approccio garantisce trasparenza, sicurezza e una riduzione dei costi operativi, eliminando la necessità di intermediari tradizionali e consentendo agli utenti di partecipare direttamente al processo di compravendita.

La documentazione di questo progetto è organizzata come segue:

Architettura del Sistema Saranno presentate l'architettura e le tecnologie utilizzate, con particolare attenzione al backend e alla Blockchain.

Analisi del Perimetro di Attacco Verranno delineati il perimetro di attacco e le principali minacce.

Possibili Attacchi e Come Difendersi Infine, saranno discussi i possibili attacchi e le contromisure adottate per mitigarli.

Capitolo 2

Architettura del Sistema

L'architettura di **BuySellChain** adotta un approccio ibrido che integra la flessibilità del Web 2.0 con la sicurezza del Web 3.0. Il sistema è progettato per garantire la disintermediazione della compravendita immobiliare, come richiesto dagli obiettivi di progetto.

Frontend: Interfaccia utente realizzata con **Jinja2** e **Alpine.js**, che fornisce reattività lato client senza richiedere una build pipeline complessa. La responsabilità è limitata alla presentazione dei dati e all'interazione utente, senza contenere logica di business critica.

Backend (Security Gateway): Server applicativo implementato in **Python** con il microframework **Flask**, che funge da middleware di sicurezza tra il client pubblico e la rete privata Blockchain. Questa componente ha il ruolo critico di sanitizzare gli input, gestire le sessioni, implementare il controllo degli accessi basato su ruoli (RBAC) e prevenire l'esposizione diretta delle API del Ledger. Si occupa dell'autenticazione degli utenti tramite credenziali verificate sul database off-chain e della generazione di JWT per l'autorizzazione stateless delle operazioni successive.

Blockchain Layer: Il nucleo del sistema, basato su **Hyperledger Fabric** [1], che funziona come database distribuito e immutabile per le operazioni critiche di compravendita. Questo livello è accessibile esclusivamente tramite il client Lisp fornito dal docente, che incapsula la logica di interazione con il chaincode e la gestione delle identità crittografiche (MSP). Il meccanismo di consenso distribuito valida le transazioni secondo le regole definite negli Smart Contract, garantendo l'integrità e la non-ripudiabilità delle operazioni.

Database Off-Chain: Un database relazionale **PostgreSQL** utilizzato per la persistenza dei dati sensibili e non critici per il consenso distribuito, come i profili utente, le credenziali di accesso al portale web e l'anagrafe del sistema. L'accesso è mediato attraverso **SQLAlchemy**, un'astrazione ORM che garantisce protezione contro attacchi SQL injection. Questa separazione garantisce la conformità alle normative sulla privacy (GDPR), evitando la scrittura di dati sensibili sul Ledger immutabile. Le password sono protette mediante l'algoritmo di hashing bcrypt con salt unico per ogni utente.

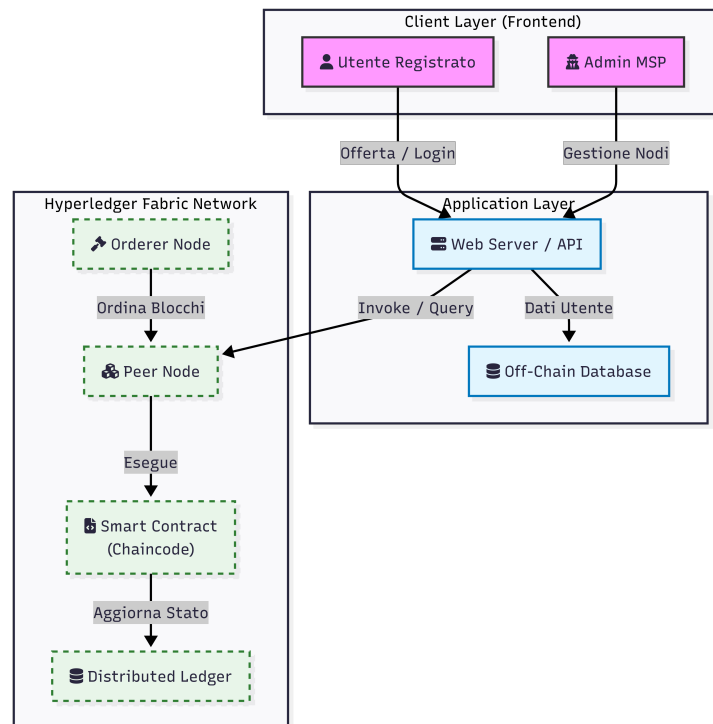


Figura 2.1: Schema ad alto livello dell'architettura ibrida Web 2.0 / Web 3.0

2.1 Gestione dell'Identità e Sicurezza

Poiché Hyperledger Fabric è una rete di tipo *permissioned*, a differenza delle blockchain pubbliche, tutti i partecipanti devono essere identificati e autorizzati prima di poter interagire con il ledger. In questo contesto, la fiducia non è anonima ma basata su identità crittografiche verificate.

Per gestire questo livello di autorizzazione, l'architettura si basa su un componente fondamentale:

Definizione (*Membership Service Provider (MSP)*). *L'MSP è il componente astratto del sistema che definisce le regole di validazione delle identità. Esso agisce come un'autorità di certificazione logica che consente alla rete di riconoscere e considerare affidabili i certificati digitali presentati dai peer e dagli utenti.*

Il funzionamento operativo coinvolge la cooperazione tra MSP e le *Certificate Authorities (CA)*:

1. **Rilascio dell'Identità:** Le CA generano una coppia di chiavi (pubblica e privata) e rilasciano un certificato X.509 che rappresenta l'identità digitale dell'attore.
2. **Verifica (Ruolo dell'MSP):** Quando un attore (es. un Peer) firma digitalmente una transazione usando la propria chiave privata, l'MSP interviene per verificare che tale certificato sia stato emesso da una CA fidata e che l'attore possieda i permessi necessari per quella specifica operazione (es. avallare una transazione).

In questo scenario, il client Lisp incapsula tale processo; di conseguenza l'attenzione è rivolta all'autenticazione dell'utente presso il server Web (Security Gateway).

2.2 User Requirements Definition

In questa sezione vengono definiti gli attori che interagiscono con la piattaforma **BuySell-Chain** e le azioni a loro consentite. Il sistema è progettato per operare in un ambiente *trustless* basato su Hyperledger Fabric, eliminando la necessità di intermediari umani nella fase di negoziazione.

User Classes and Characteristics

Amministratore di Sistema: Gestisce la piattaforma, monitora anomalie e alert di sicurezza. Dispone di una dashboard dedicata per osservabilità, auditing e interventi correttivi.

Utente Registrato: Attore abilitato alla compravendita; lo stesso account può ricoprire ruoli distinti:

- **Venditore (Proprietario dell'asset):** Possiede l'immobile digitalizzato sul registro e avvia l'asta definendone i parametri.
- **Offerente (Bidder):** Partecipa all'asta inviando offerte firmate e vincolanti secondo le regole dello Smart Contract.

Utente Non Autenticato: Visitatore esterno. Senza account può consultare le aste e gli immobili pubblicati, senza possibilità di interazione avanzata.

User Actions and Use Cases

Le azioni riflettono la logica di business gestita dal server Web e dagli Smart Contract.

- **Azioni Utente Non Autenticato:**
 - Consultare la landing page del progetto.
 - Registrarsi o accedere.
 - Effettuare un login sicuro.
- **Azioni dell'Amministratore (Monitoraggio e Sicurezza):**
 - Monitorare gli alert su tentativi di violazione dell'integrità (mismatch crittografici).
 - Analizzare i log delle offerte rifiutate per violazioni delle finestre temporali (anti-frontrunning).
- **Azioni dell'Utente Registrato — Venditore:**
 - Avviare un'asta per un immobile già digitalizzato.
 - Impostare la finestra temporale dell'asta (inizio/fine) gestita dallo Smart Contract.

- Visualizzare in tempo reale sul Ledger lo stato delle offerte ricevute.
- Ricevere la notifica di aggiudicazione e il trasferimento della proprietà digitale.
- **Azioni dell'Utente Registrato — Offerente:**
 - Consultare le aste immobiliari attive.
 - Inviare un'offerta firmata; lo Smart Contract verifica l'incremento minimo richiesto.
 - Verificare la propria posizione in graduatoria (anonimizzata o pubblica secondo la logica di business).
 - Visualizzare l'esito della validazione (accettata/rifiutata per motivi temporali o di importo).

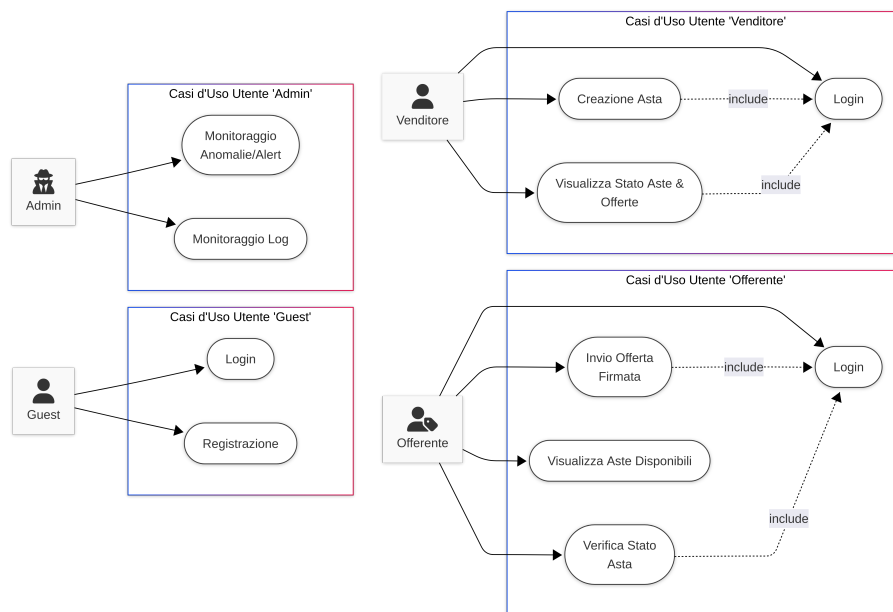


Figura 2.2: Azioni degli utenti all'interno della piattaforma

2.3 Frontend

Per la realizzazione del frontend è stato adottato il motore di templating **Jinja2** [2], integrato nativamente in Flask, per la generazione dinamica delle pagine HTML lato server. Jinja2 consente di creare template modulari e riutilizzabili, separando la logica di presentazione dai dati applicativi e facilitando la manutenzione del codice.

Per l'interattività lato client, è stato scelto il framework leggero **Alpine.js** [3], che fornisce reattività senza richiedere una build pipeline complessa. Alpine.js permette di dichiarare il comportamento dinamico direttamente nel markup HTML mediante direttive semplificate, risultando ideale per applicazioni web moderne ma senza la complessità di framework come React o Vue.

Questa combinazione garantisce un frontend performante, mantenibile e facilmente integrabile con il backend Flask, preservando la semplicità dell'architettura complessiva.

Il frontend consiste delle seguenti route:

/login Pagina di autenticazione per l'accesso degli utenti registrati.

/signin Pagina di registrazione per i nuovi utenti.

/ Pagina home nella quale vengono presentate tutte le aste disponibili in formato grid. Ogni card della griglia mostra la foto principale dell'immobile, il titolo dell'asta, il prezzo di partenza e lo stato. Gli utenti non autenticati possono consultare le aste ma non partecipare.

/faq Pagina statica con le domande frequenti e le informazioni di contatto per l'assistenza.

/auctions/create Pagina per la creazione di una nuova asta. Accessibile solo dai venditori che abbiano precedentemente registrato almeno un immobile. Consente di impostare i parametri temporali (data inizio e fine), il prezzo di partenza e l'incremento minimo.

/auctions/{id} Route dinamica generata per ciascun'asta. Visualizza i dettagli completi dell'immobile, lo stato dell'asta in tempo reale, la graduatoria delle offerte (ove applicabile) e consente agli utenti autenticati di partecipare inviando un'offerta.

/profile Pagina del profilo utente nella quale sono disponibili i dati personali, la cronologia delle aste create (per i venditori) e la cronologia delle offerte inviate (per gli offerenti).

2.4 Backend (Security Gateway)

In questa sezione verrà analizzato il backend insieme alla logica di business. Questa sezione descrive nel dettaglio il componente architetturale che funge da ponte sicuro tra l'interfaccia pubblica (Frontend) e la rete privata (Blockchain Layer). Il Security Gateway non si limita a inoltrare le richieste, ma implementa la logica applicativa necessaria per garantire l'integrità del sistema, la gestione delle identità e la validazione preliminare delle transazioni, proteggendo il Ledger da accessi non autorizzati o malformati.

La tecnologia scelta per l'implementazione del backend è **Python** [4], utilizzando il microframework **Flask** [5, 6]. Questa scelta è motivata dalla necessità di avere un ambiente di sviluppo rapido, flessibile e con un ecosistema ricco di librerie per la crittografia e la gestione delle reti. Il server Flask espone una serie di API RESTful che vengono consumate dal client Frontend.

2.4.1 Authentication & Authorization

La sicurezza di **BuySellChain** si articola su due livelli distinti ma interconnessi: l'autenticazione degli utenti e la gestione dei permessi operativi.

- **Autenticazione (Identity Verification):** Il Security Gateway agisce come punto d'ingresso unico. Verifica le credenziali dell'utente (username e password) consultando il database off-chain. Una volta validata l'identità, il sistema mappa l'utente Web 2.0 a un'identità digitale riconosciuta dalla rete Fabric, garantendo che ogni operazione sul ledger sia tracciabile e attribuibile a un attore specifico.
- **Autorizzazione (Access Control):** Dopo l'autenticazione, il sistema applica politiche di controllo degli accessi basate sui ruoli (RBAC). Il backend verifica se l'utente possiede i privilegi necessari per invocare specifiche transazioni (es. solo un utente con ruolo "Venditore" può inizializzare un'asta).

Il Security Gateway funge da *Policy Enforcement Point*, intercettando le richieste e verificando che il richiedente possieda i privilegi necessari (es. solo il proprietario di un asset può metterlo all'asta) prima di inoltrare qualsiasi comando alla Blockchain.

Una volta completata con successo la fase di login, il server genera e firma un *JSON Web Token* (JWT) [7]. Questo token funge da passaporto digitale per l'utente e contiene, nel suo *payload*, informazioni essenziali quali l'identificativo utente, i ruoli assegnati e la data di scadenza della sessione.

Il client (Frontend) riceve il token e lo memorizza in modo sicuro. Per ogni successiva interazione con le API del sistema, il client deve includere il JWT nell'header della richiesta HTTP (solitamente nell'header **Authorization: Bearer <token>**). Questo meccanismo *stateless* permette al backend di validare l'identità del richiedente e verificare le autorizzazioni per ogni singola operazione senza dover interrogare continuamente il database delle credenziali.

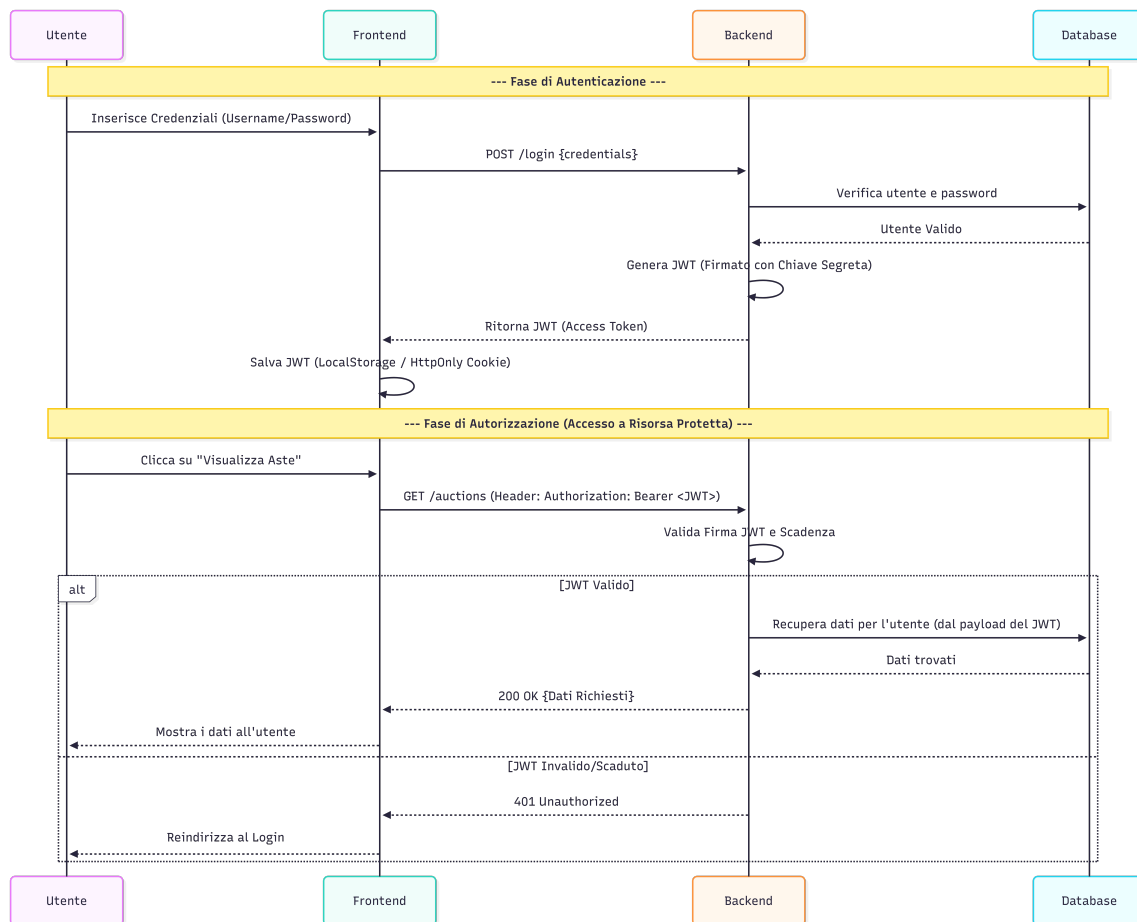


Figura 2.3: Flusso di autenticazione/autorizzazione

2.4.2 Modello Dati & Interfaccia RESTful API

Poiché Hyperledger Fabric mantiene il world state come mappa di coppie (**key**, **value**), a livello applicativo è necessario definire: convenzioni di naming per le chiavi (namespace e identificatori), classi/entità applicative e un formato di serializzazione deterministico (es. JSON).

Le API di base fornite dal prof. Arcieri sono pubblicate come endpoint REST. Il Security Gateway riceve le richieste HTTP della piattaforma web e le traduce in comandi per il client Lisp verso la rete Fabric. Ad esempio, una chiamata HTTP `POST /api/v1/make_bid` viene mappata al comando `AddKV`. Il client invoca quindi le funzioni di *chaincode* per creare, leggere, aggiornare o eliminare coppie (**key**, **value**) sul ledger.

Le primitive disponibili sono:

AddKV: Inserisce una nuova coppia (**key**, **value**) in una classe applicativa.

GetKV: Restituisce il **value** associato a una **key** di una classe.

GetKeyHistory: Restituisce la cronologia immutabile delle operazioni su una **key**.

DelKV: Elimina la coppia (**key**, **value**) associata a una **key**.

GetClasses: Restituisce l'elenco delle classi disponibili.

GetNumKeys: Restituisce il numero di coppie presenti in una classe.

GetKeys: Restituisce l'elenco delle chiavi presenti in una classe.

Listing 2.1: Envelope di richiesta alle API base

```
1 {  
2   "cmd": "AddKV | GetKV | GetKeyHistory | DelKV | GetClasses |  
        GetNumKeys | GetKeys",  
3   "class": "NomeClasse",  
4   "key": "Key",  
5   "value": { ... }  
6 }
```

Definizione Modello Dati

Di seguito vengono presentate le definizioni strutturali delle classi dati che costituiscono il modello di persistenza distribuito. Ogni tabella rappresenta un'entità logica memorizzata in modo immutabile sulla Blockchain, specificando i campi, i tipi di dato e le relazioni chiare necessarie per garantire l'integrità referenziale e la tracciabilità delle operazioni di compravendita immobiliare all'interno del Ledger di Hyperledger Fabric.

Campo	Tipo	Note
id	string	Identificativo univoco
ownerId	string	Riferimento all'utente proprietario
title	string	Titolo descrittivo immobile
location	string	Indirizzo fisico
description	string	Descrizione immobile
currentAuctionId	string null	ID dell'asta attiva (se presente)
createdAt	int (epoch)	Data creazione
status	enum	ACTIVE, LOCKED, TRANSFERRED

Tabella 2.1: Definizione della Classe Asset

Campo	Tipo	Note
id	string	Identificativo univoco offerta
auctionId	string	Riferimento all'asta
bidderId	string	Riferimento all'offerente
amount	number	Importo offerto
timestamp	int (epoch)	Data e ora offerta
status	enum	APPROVED, REJECTED
rejectReason	string null	Motivo del rifiuto (se applicabile)

Tabella 2.2: Definizione della Classe Bid

Campo	Tipo	Note
id	string	Identificativo univoco dell'asta
assetId	string	Riferimento all'asset in vendita
sellerId	string	Riferimento al venditore
startTime	int (epoch)	Data inizio asta
endTime	int (epoch)	Data fine asta
startPrice	number	Prezzo di partenza
minIncrement	number	Incremento minimo rilancio
highestBidId	string null	ID dell'offerta vincente attuale
highestBidAmount	number null	Importo dell'offerta vincente
bidsCount	int	Contatore totale offerte
status	enum	ACTIVE, CLOSED, CANCELLED, LOCKED

Tabella 2.3: Definizione della Classe Auction

Pertanto, ipotizzando un'analogia con le *tabelle SQL* per fini rappresentativi, la struttura delle classi può essere visualizzata come nello schema seguente:

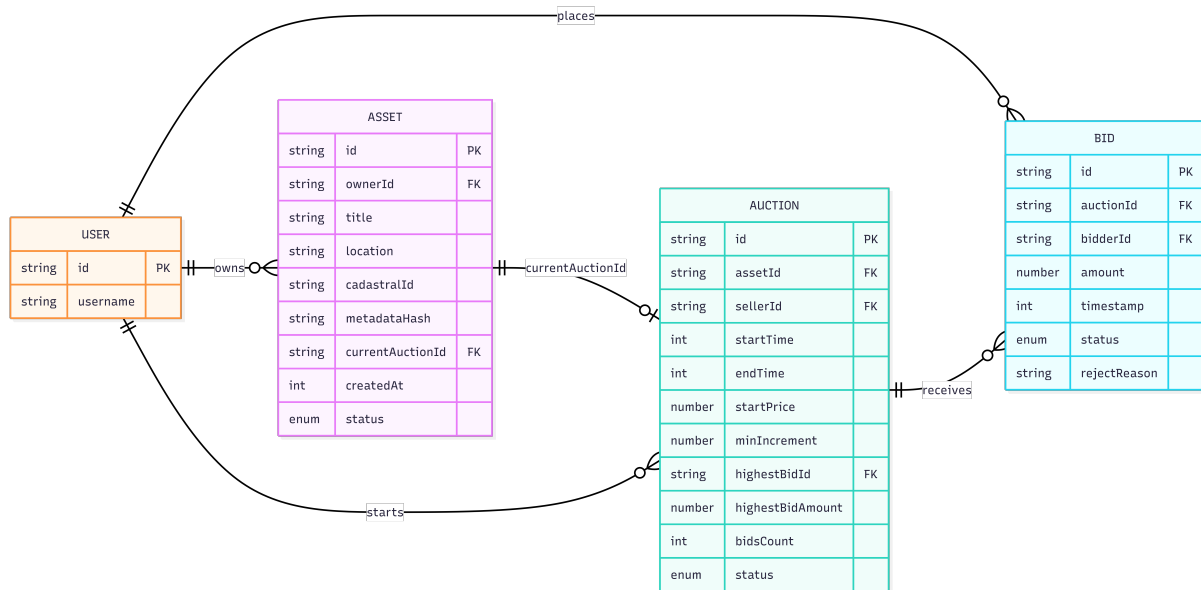


Figura 2.4: Schema logico delle tabelle su Fabric

La figura illustra le relazioni logiche tra le entità. L'utente (User) è l'attore principale: può possedere molteplici Asset, avviare diverse Auction e partecipare attivamente inviando Bid. Gli Asset possono essere associati a una Auction attiva (relazione 1:1 opzionale), mentre ogni Auction è legata univocamente a un bene specifico e funge da contenitore per tutte le offerte ricevute.

Questo modello relazionale, pur essendo implementato su un archivio chiave-valore non relazionale, è progettato per garantire la completa tracciabilità della filiera di vendita (provenance). Mantenendo riferimenti incrociati tra venditori, offerenti, beni e stati

delle aste, il sistema permette di ricostruire in modo deterministico e immutabile l'intera cronologia di una transazione immobiliare, dall'inserimento dell'asset fino all'aggiudicazione finale, assicurando così la massima trasparenza e integrità dei dati.

Definizione Interfaccia RESTful API

Definita la struttura logica dei dati, il passo successivo consiste nell'esposizione dell'interfaccia RESTful API. Nella fase attuale di sviluppo (sperimentale), il server Web è configurato per l'ascolto sulla porta locale standard 5000. L'endpoint base per tutte le richieste API è:

`http://localhost:5000/api/v1/`

Per ridurre la complessità infrastrutturale in ambiente di test, la comunicazione avviene attualmente su protocollo HTTP non cifrato.

I principali endpoint sono:

POST /auth/signin Registrazione di un nuovo utente. La richiesta crea un'identità sul database off-chain.

POST /auth/login Autenticazione utente. Verifica le credenziali e restituisce il token JWT necessario per autorizzare le chiamate successive.

POST /assets Registrazione di un nuovo immobile sul ledger. Richiede che l'utente sia autenticato e abbia il ruolo di "Venditore". Internamente invoca **AddKV** per la classe **Asset**.

GET /assets Recupero dettagli di tutti gli assets. Internamente invoca **GetKeys** e **GetKV** per la classe omonima.

GET /assets/{id} Recupero dettagli di uno specifico asset. Internamente invoca **GetKV** per la classe omonima.

GET /assets/user Recupero dettagli di tutti gli asset posseduti dall'utente autenticato. Internamente invoca **GetKeys** e **GetKV** per la classe **Asset**, filtrando i risultati in base all'ID del proprietario recuperato tramite **JWT**.

POST /auctions Creazione di una nuova asta per un asset posseduto dall'utente. Il sistema verifica la proprietà dell'asset e imposta lo stato iniziale dell'asta. Invoca **AddKV** per la classe **Auction**.

GET /auctions Elenco di tutte le aste presenti a sistema. Internamente utilizza **GetKeys**

GET /auctions/status/{status} Filtraggio delle aste per stato (es. **ACTIVE**, **CLOSED**). Internamente utilizza **GetKeys** e **GetKV** per la classe **Auction**.

GET /auctions/{id} Recupero dettagli di una specifica asta. Internamente invoca **GetKV** per la classe omonima.

GET /auctions/bids/{id} Recupero di tutte le offerte di una specifica asta. Internamente invoca **GetKeys** e **GetKV** per la classe **Bids**, filtrando i risultati in base all'ID dell'asta.

DELETE /auctions/{id} Cancellazione di una specifica asta. Internamente invoca **DelKV** per la classe omonima.

- POST /bids** Invio di un'offerta per un'asta attiva. La logica di backend verifica che l'asta sia aperta, che l'offerta superi l'importo corrente più l'incremento minimo, e che l'offerente non sia il venditore stesso. Se valida, registra l'offerta tramite **AddKV** sulla classe **Bid** e aggiorna lo stato dell'asta.
- GET /bids/user** Recupera tutte le offerte inviate dall'utente autenticato. Internamente invoca **GetKeys** e **GetKV** per la classe **Bid**, filtrando i risultati in base all'ID dell'offerente recuperato tramite **JWT**.
- GET /bids/latest/{id}** Recupero dell'ultima offerta valida fatta dall'utente per una specifica asta. Internamente invoca **GetKeys** e **GetKV** per la classe **Bid**, filtrando i risultati in base all'ID dell'asta.
- GET /admin/users** Restituisce l'elenco completo degli utenti registrati nel database off-chain e il loro stato (attivo/sospeso), per permettere all'amministratore di gestire le utenze.
- GET /admin/stats** Fornisce metriche aggregate del sistema, come il numero totale di aste concluse, il volume delle transazioni e il numero di asset registrati, invocando **GetNumKeys** sulle diverse classi.
- GET /admin/history/{key}** Endpoint di auditing che espone la cronologia immutabile di una specifica chiave (asset o asta), invocando la primitiva **GetKeyHistory** del chaincode.

Listing 2.2: Esempio di richiesta AddKV per aggiungere un Asset

```
1  "cmd": "AddKV",
2  "class": "Assets",
3  "key": [
4      "ef25bc38bb1182a80bd7d4b01bc45d9a34/"
5      "b84c939bf944459dc44a79c0bf6259"
6  ],
7  "value": {
8      "asset_type": "villa",
9      "created_at": "2026-03-11T11:59:41.544210",
10     "current_auction_id": null,
11     "description": "Villetta indipendente",
12     "id": "ef25bc38bb1182a80bd7d4b01bc45d/"
13     "9a34b84c939bf944459dc44a79c0bf6259",
14     "location": "Roma",
15     "owner_id": "32503db9ad32a1b8999988977b40d/"
16     "fad523ec82a8a1251ce8c05c85842471b0c",
17     "price": 50000,
18     "size": 120,
19     "status": "active",
20     "title": "Villa"
21 }
```

Listing 2.3: Esempio di richiesta AddKV per creare un'asta

```
1  {
2      "cmd": "AddKV",
3      "class": "Auctions",
4      "key": "1781938b47129ab1e6059a4ec5cca/"
5      "f5336e69406de0852683e765c799cee7d04",
6      "value": {
7          "id": "1781938b47129ab1e6059a4ec5cca/"
8          "f5336e69406de0852683e765c799cee7d04",
9          "assetId": "ef25bc38bb1182a80bd7d4b01bc45d/"
10         "9a34b84c939bf944459dc44a79c0bf6259",
11         "asset_title": "Villa",
12         "asset_description": "Villetta indipendente",
13         "sellerId": "32503db9ad32a1b8999988977b40dfad/"
14         "523ec82a8a1251ce8c05c85842471b0c",
15         "bid_count": 0,
16         "high_bid_amount": null,
17         "high_bid_id": null,
18         "image_url": "/static/uploads/assets/seller_id/asset_id/primary
19             -8b97d3c3f46f4546ac9c8407496901cb.png",
20         "min_incr": 15000,
21         "start_time": "2026-03-11T15:00:00",
22         "end_time": "2026-03-11T16:00:00",
23         "starting_price": 25000,
24         "status": "ACTIVE"
25     }
```

Listing 2.4: Esempio di richiesta AddKV per aggiungere un'offerta (Bid)

```
1 {
2   "cmd": "AddKV",
3   "class": "Bids",
4   "key": "8f9b21b63558e4154fd8f0bf8cbaf7b/
5   0aaa7170dbd7e0565e0af60ffc782ecd4",
6   "value": {
7     "id": "8f9b21b63558e4154fd8f0bf8cbaf7b/
8     0aaa7170dbd7e0565e0af60ffc782ecd4",
9     "auctionId": "1781938b47129ab1e6059a4ec5cca/
10    f5336e69406de0852683e765c799cee7d04",
11    "bidderId": "01b1de7136182f016755e0bac1a61f60/
12    92df799e9fea2bf9848e1c682a73c9cf",
13    "bid_amount": 80000,
14    "timestamp": 2026-03-11T12:11:00.764654,
15    "status": "approved",
16    "reason": null
17  }
18 }
```

2.4.3 Logica di Bussiness

La piattaforma implementa un modello di gestione degli attori e delle aste articolato in fasi distinte:

Registrazione e Verifica dei Venditori Un utente che desideri partecipare alle aste come venditore deve completare una procedura di registrazione qualificata. Oltre alle informazioni anagrafiche standard (nome, cognome, email, numero di telefono), il sistema richiede l'inserimento del codice fiscale, necessario per garantire l'identificazione univoca e conforme alle normative sulla tracciabilità delle operazioni immobiliari. Una volta sottomessa la richiesta, un messaggio di notifica viene inviato agli amministratori del sistema, i quali hanno la responsabilità di validare l'identità del richiedente e approvare o rifiutare l'iscrizione al ruolo di venditore.

Struttura Temporale delle Aste Ogni asta è caratterizzata da un intervallo temporale definito dal venditore, durante il quale gli offerenti possono partecipare. Tale intervallo è suddiviso logicamente in finestre temporali di durata predeterminata, al fine di implementare meccanismi anti-frontrunning e garantire equità nella partecipazione.

Vincoli sulle Offerte Per ogni finestra temporale, ciascun offerente può inviare al massimo una sola offerta. Questa limitazione impedisce comportamenti speculativi e riduce la complessità computazionale del sistema. Le offerte rimangono nascoste agli altri partecipanti fino al termine della relativa finestra temporale, garantendo così l'anonimato del processo di bidding.

Aggiornamento dello Stato dell'Asta Al termine di ogni finestra temporale, il sistema esegue automaticamente un'operazione di consolidamento dello stato dell'asta:

- Vengono valutate tutte le offerte ricevute durante la finestra.

- I campi `highestBidAmount` e `bidsCount` vengono aggiornati per riflettere l'offerta vincente e il conteggio totale.
- Lo stato dell'asta transisce in `LOCKED`, impedendo temporaneamente l'invio di nuove offerte.
- Il venditore acquisisce la possibilità di modificare il parametro `minIncrement`, adattando dinamicamente il valore richiesto per i successivi rilanci.

Una volta completato l'aggiornamento, l'asta ritorna allo stato `ACTIVE`, rendendo di nuovo disponibile la partecipazione per la finestra temporale successiva.

La finestra temporale dell'asta viene definita dagli admin, ed è preimpostata a 12 ore (dalle 8.00 AM fino alle 20.00 PM).

Questo meccanismo garantisce una distribuzione equa degli intervalli di partecipazione e facilita l'implementazione di politiche anti-frontrunning, in quanto ogni finestra temporale costituisce un perimetro di validazione indipendente per le offerte ricevute.

2.5 Blockchain Layer

L'ultimo layer dell'architettura è rappresentato dalla blockchain Hyperledger Fabric, che svolge due ruoli fondamentali nel sistema: fungere da database distribuito e immutabile, e validare le transazioni effettuate dagli utenti secondo le regole definite negli Smart Contract.

In questo contesto, Fabric agisce come sistema di registrazione ufficiale (ledger) per tutte le operazioni critiche di compravendita: dalla registrazione degli asset immobiliari, alla creazione delle aste, fino alla registrazione delle offerte. La natura distribuita e permissioned della rete garantisce che ogni transazione sia validata da più peer secondo il meccanismo di consenso configurato, assicurando così l'integrità e la non-ripudiabilità delle operazioni.

Il Security Gateway comunica con la rete Fabric esclusivamente tramite il client Lisp fornito dal docente, che incapsula la logica di interazione con il chaincode e la gestione delle identità crittografiche (MSP). Questo isolamento protegge il ledger da accessi diretti e non autorizzati.

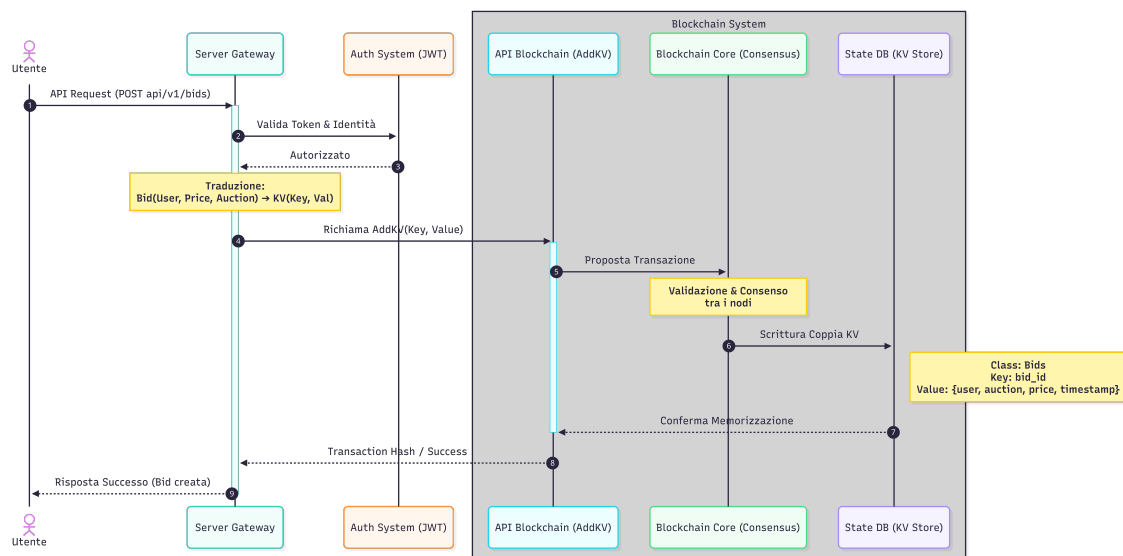


Figura 2.5: Flusso di azioni per la creazione di un'offerta

2.6 Database Off-Chain

Il database off-chain è responsabile della persistenza dei dati sensibili e delle informazioni anagrafiche degli utenti, garantendo che tali dettagli non vengano esposti direttamente sul Ledger immutabile. Il Security Gateway interagisce con questo componente per validare le credenziali di accesso e gestire i permessi, fungendo da ponte tra l'identità dell'utente sulla piattaforma web e il suo corrispettivo crittografico sulla rete Blockchain. La tecnologia adottata per l'implementazione è PostgreSQL [8].

Campo	Tipo	Note
id	string	allineato a <code>user:<uuid></code> on-chain
role	array di enum	ADMIN, SELLER, BIDDER
name	string	nome utente
surname	string	cognome utente
email	string	indirizzo email
birthday	date	data di nascita
cellularNumber	number	numero di cellulare
passwordHash	string	hash della password
lastLoginAt	int (epoch)	ultima data di login
createdAt	int (epoch)	data di creazione account

Tabella 2.4: Tabella Users

Per l'interazione con PostgreSQL nel contesto del framework Flask, è stata adottata la libreria **SQLAlchemy** [9], un'astrazione Object-Relational Mapping (ORM) che consente di operare su query strutturate anziché ricorrere a SQL puro. Questo approccio garantisce una maggiore sicurezza nei confronti di attacchi SQL injection e una migliore manutenibilità del codice.

Al fine di semplificare la gestione dell'infrastruttura di sviluppo e garantire la portabilità dell'applicazione, il database PostgreSQL è stato containerizzato mediante **Docker**. La configurazione del container è definita in un file `docker-compose.yml`, il quale specifica i parametri di connessione, il volume di persistenza dei dati e le variabili di ambiente necessarie. Questo approccio consente agli sviluppatori di avviare l'intera stack del database con un singolo comando, eliminando le dipendenze dall'ambiente locale e facilitando il deployment su sistemi eterogenei.

Per la messa in sicurezza delle credenziali è stato adottato l'algoritmo di hashing crittografico **bcrypt**, basato sul cifrario Blowfish. Il processo integra nativamente la generazione di un salt univoco per ogni utente, restituendo una stringa finale che concatena versione, costo, salt e digest secondo la struttura:

$$\text{pwd_hash} = \$2b\$[\text{cost}][\$[\text{salt}]]\$[\text{hash}]$$

Questo approccio garantisce protezione contro attacchi di tipo rainbow table e attacchi a forza bruta grazie al fattore di costo adattivo.

Capitolo 3

Superficie osservabile e Perimetro di Attacco

L'architettura di `BuySellChain` adotta un approccio ibrido che integra la flessibilità del Web 2.0 con la sicurezza decentralizzata del Web 3.0. Questa disaccoppiatura tra il Security Gateway, il database off-chain e il ledger distribuito su Hyperledger Fabric offre robuste garanzie architettoniche, ma espande parallelamente la superficie d'attacco. Un potenziale avversario potrebbe sfruttare vulnerabilità in diversi punti di contatto tra questi layer. Di seguito viene delineato in dettaglio il perimetro d'attacco, analizzando i possibili vettori di minaccia per ciascun livello dell'infrastruttura.

3.1 Livello Client (Frontend)

Il frontend, realizzato con i template Jinja2 e il framework Alpine.js, rappresenta il punto di interazione primario per gli utenti autenticati e non autenticati. Sebbene non contenga logica di business critica, è esposto a vulnerabilità tipiche delle applicazioni web lato client:

- **Cross-Site Scripting (XSS):** Poiché il frontend genera pagine HTML dinamiche tramite Jinja2, un'inadeguata sanitizzazione degli input dell'utente (ad esempio nella descrizione di un immobile o nei parametri di registrazione) potrebbe permettere a un attaccante di iniettare script JavaScript malevoli. Se eseguiti nel browser di un'altra vittima, questi script potrebbero sottrarre informazioni sensibili.
- **Furto del Token JWT e Session Hijacking:** A seguito dell'autenticazione, il token JWT viene salvato localmente nel client (LocalStorage o Cookie). Se un attacco XSS dovesse avere successo e il token fosse conservato nel LocalStorage (che non è protetto dal flag `HttpOnly`), l'attaccante potrebbe rubare il token ed effettuare un *session hijacking*, impersonando l'utente per compiere azioni non autorizzate (es. piazzare offerte fraudolente).
- **Manipolazione dell'Interfaccia e CSRF:** Un utente malintenzionato potrebbe alterare il DOM locale (ad esempio riattivando pulsanti disabilitati per aste chiuse) nel tentativo di inviare payload imprevisti al server. Inoltre, in assenza di header `Anti-CSRF`, richieste forgiate da domini terzi potrebbero ingannare il browser dell'utente portandolo a eseguire azioni non volute.

3.2 Livello Application / Security Gateway (Backend)

Il server applicativo basato su Flask svolge il ruolo di middleware e Policy Enforcement Point. Essendo l'unico componente direttamente esposto alla rete pubblica per filtrare le richieste dirette al ledger, rappresenta il bersaglio primario:

- **Intercettazione Man-in-the-Middle (MiTM):** Nell'attuale configurazione di sviluppo, il server ascolta sulla porta locale 5000 utilizzando il protocollo HTTP non cifrato. Questo espone integralmente il traffico di rete: credenziali in chiaro durante la `POST /auth/login` e token JWT trasmessi nell'header `Authorization` sono facilmente intercettabili tramite *packet sniffing*.
- **Bypass dei Controlli di Autorizzazione (RBAC) e Vulnerabilità JWT:** Il sistema fa forte affidamento sui JWT per l'autorizzazione stateless. Se la chiave segreta (HMAC) utilizzata per firmare i token risultasse debole o venisse compromessa, un attaccante potrebbe forgiare token validi alterando il payload per elevare i propri privilegi (ad esempio, assegnandosi il ruolo `SELLER` o `ADMIN`). Inoltre, vulnerabilità nell'implementazione della libreria potrebbero permettere attacchi di tipo *signature stripping* (es. forzare l'algoritmo `none`).
- **Lisp Command Injection e Parameter Tampering:** Il backend traduce le richieste RESTful in primitive per il client Lisp (`AddKV`, `GetKV`, `DelKV`). Qualora la validazione dei JSON in ingresso da parte dell'API fosse carente, un attaccante potrebbe inviare payload malformati (es. chiavi contenenti caratteri di escape non previsti o strutture annidate anomale) tentando di manipolare la query destinata al chaincode o di causare un Denial of Service (DoS) nel parser Lisp.
- **Abuso delle API e Denial of Service (DoS):** L'assenza esplicita di meccanismi di *Rate Limiting* sugli endpoint pubblici (come `/api/v1/auth/login` o `/api/v1/bids`) permette ad attaccanti di sferrare attacchi di tipo Brute Force contro gli account degli utenti, o di inondare il sistema di offerte fasulle saturando le risorse di calcolo del Security Gateway e rallentando il consenso della blockchain.

3.3 Livello Dati Off-Chain (Database PostgreSQL)

Il database PostgreSQL containerizzato gestisce dati anagrafici sensibili (GDPR) e credenziali:

- **SQL Injection Avanzato:** L'uso dell'ORM SQLAlchemy mitiga gran parte dei rischi di SQL injection standard. Tuttavia, se in qualche punto del codice sorgente venissero utilizzate query SQL raw senza un'adeguata sanitizzazione, un attaccante potrebbe sfruttare questa vulnerabilità per esfiltrare dati sensibili, modificare record o addirittura eseguire comandi arbitrari sul server database.
- **Cracking Off-Line delle Credenziali:** Le password sono protette da hashing `bcrypt` con salt unico. Se il parametro di costo adattivo (cost factor) fosse configurato su un valore troppo basso per favorire le performance, un attaccante che riuscisse a ottenere un dump del database potrebbe eseguire efficaci attacchi dizionario offline utilizzando hardware dedicato (GPU/ASIC).

- **Vulnerabilità dell'Ambiente Docker (Container Escape):** Il database è distribuito tramite `docker-compose.yml`. Misconfigurazioni operative, come l'esecuzione del container in modalità *privileged*, l'eccessiva esposizione delle porte (es. esporre la porta 5432 globalmente su 0.0.0.0 anziché solo al backend), o vulnerabilità non patchate nell'engine di Docker, potrebbero consentire a un aggressore di sfuggire all'isolamento del container e compromettere l'intero server host.

Capitolo 4

Possibili Attacchi e Contromisure

Alla luce della superficie d'attacco analizzata nel capitolo precedente, è fondamentale definire una strategia di difesa in profondità (*Defense-in-Depth*). Il sistema `BuySellChain` deve garantire la confidenzialità, l'integrità e la disponibilità dei dati in ogni layer della sua architettura ibrida. Di seguito vengono descritti in dettaglio gli scenari di attacco più critici e le contromisure tecniche e architetturali adottate per la loro mitigazione.

4.1 Difesa del Frontend e Gestione Sicura delle Sessioni

Il layer di presentazione, pur essendo privo di logica di business, rappresenta il primo punto di contatto con l'utente ed è esposto ad attacchi client-side.

- **Attacco: Cross-Site Scripting (XSS) e Furto di JWT**
Come evidenziato, l'archiviazione del token JWT nel `LocalStorage` del browser espone il sistema al furto delle sessioni in caso di vulnerabilità XSS.
- **Contromisura: Cookie `HttpOnly` e CSP**
Per mitigare questo rischio, il Security Gateway è configurato per rilasciare il token JWT esclusivamente tramite un *Cookie* di sessione protetto dai flag `HttpOnly`, `Secure` e `SameSite=Strict`. Il flag `HttpOnly` rende il cookie invisibile agli script JavaScript eseguiti nel browser (es. `Alpine.js` o script malevoli), impedendone il furto diretto. Inoltre, viene implementata una rigorosa *Content Security Policy* (CSP) tramite gli header HTTP per restringere le fonti da cui il browser è autorizzato a caricare script esterni, riducendo drasticamente la probabilità di successo di un attacco XSS.
- **Attacco: Cross-Site Request Forgery (CSRF)**
L'utilizzo di cookie di sessione reintroduce il rischio di CSRF, dove un sito malevolo potrebbe forzare il browser dell'utente a inviare richieste autenticate al backend di `BuySellChain`.
- **Contromisura: Token Anti-CSRF**
Viene introdotto un meccanismo di token sincrono (*Synchronizer Token Pattern*). Il server Flask genera un token CSRF univoco per ogni sessione, integrato nei form

HTML generati da Jinja2. Ogni richiesta di modifica dello stato (es. `POST /bids`) deve includere questo token, validato dal backend prima di processare la transazione.

4.2 Hardening del Security Gateway (Backend)

Il backend Flask agisce come *Security Gateway* e richiede protezioni rigorose per impedire compromissioni dirette e abusi.

- **Attacco: Man-in-the-Middle (MiTM)**
L'utilizzo del protocollo HTTP in chiaro espone le credenziali e le comunicazioni API.
- **Contromisura: Cifratura TLS/SSL (HTTPS)**
In ambiente di produzione, il server Flask non viene esposto direttamente. Viene introdotto un Reverse Proxy (es. NGINX) incaricato di gestire la terminazione TLS. Tutte le comunicazioni tra il client e il sistema sono cifrate forzatamente in HTTPS (TLS 1.3), garantendo la confidenzialità in transito e neutralizzando il *packet sniffing*.
- **Attacco: Lisp Command Injection e Payload Malformati**
L'iniezione di caratteri di escape nel JSON potrebbe manipolare le query verso il client Lisp.
- **Contromisura: Validazione Rigorosa degli Input (JSON Schema)**
Prima di invocare le primitive `AddKV` o `GetKV`, il backend applica una validazione strutturale e semantica (*Input Sanitization*) tramite **JSON Schema**. Ogni campo viene tipizzato, limitato in lunghezza e filtrato per rimuovere caratteri non consentiti. Solo i payload conformi allo schema atteso vengono inoltrati alla rete Fabric.
- **Attacco: Denial of Service (DoS) e Brute Force**
Inondare gli endpoint di autenticazione o di offerta di richieste massive.
- **Contromisura: Rate Limiting e Account Lockout**
Viene implementata la libreria **Flask-Limiter** per applicare politiche di *Rate Limiting* basate sull'indirizzo IP del richiedente. Endpoint critici come `/auth/login` accettano un massimo di 5 tentativi al minuto. Superata tale soglia, l'account o l'IP subiscono un *lockout* temporaneo.

4.3 Protezione del Database Off-Chain e dell'Ambiente Docker

Il database PostgreSQL conserva dati sensibili e necessita di isolamento sia a livello applicativo che di sistema operativo.

- **Attacco: SQL Injection Avanzato**
Esecuzione di comandi SQL non previsti.
- **Contromisura: Uso esclusivo di ORM parametrizzato**
Tutte le interazioni con il database sono mediate da SQLAlchemy tramite query parametrizzate (*Prepared Statements*). È vietato da policy di sviluppo l'utilizzo di concatenazione di stringhe o query raw, annullando di fatto la fattibilità degli attacchi di tipo SQLi.

- **Attacco: Container Escape e Misconfigurazioni**

Accesso non autorizzato al file system host dal container Docker.

- **Contromisura: Principio del Minimo Privilegio (Docker Hardening)**

Il file `docker-compose.yml` è stato revisionato: il container PostgreSQL non espone la porta 5432 all'host (`ports: - "5432:5432"`), ma comunica con il backend Flask esclusivamente tramite una *Virtual Network* interna creata da Docker. Inoltre, il container viene eseguito con un utente non root e con *capabilities* del kernel ridotte (`cap_drop: - ALL`), prevenendo scalate di privilegi verso il sistema host.

Bibliografia

- [1] The Hyperledger Foundation, *Hyperledger Fabric Documentation*. The Linux Foundation, 2024. Versione 2.5 LTS. Accesso: Gennaio 2026.
- [2] A. Ronacher, “Jinja2: A full-featured template engine for Python.” <https://jinja.palletsprojects.com/>, 2026. Accessed: 2026-02-12.
- [3] C. Porzio, “Alpine.js: A rugged, minimal framework for composing JavaScript behavior in your markup.” <https://alpinejs.dev/>, 2026. Accessed: 2026-02-12.
- [4] Python Software Foundation, “Python language reference, version 3.12,” 2023. Disponibile su python.org.
- [5] Pallets Projects, “Flask: A microframework for python based on werkzeug, jinja 2 and good intentions,” 2010. Versione 3.0.x.
- [6] L. Turner, “Flask-jwt-extended: Open source extension for flask adding support for json web tokens.” <https://github.com/vimalloc/flask-jwt-extended>, 2023.
- [7] M. B. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT).” RFC 7519, May 2015.
- [8] PostgreSQL Global Development Group, *PostgreSQL: The World’s Most Advanced Open Source Relational Database*. PostgreSQL Global Development Group, 2026. Accessed: 2026-02-09.
- [9] M. Bayer, “SQLAlchemy.” <https://www.sqlalchemy.org/>, 2026. Accessed: 2026-02-12.